



ETHERNET/IP – DOCUMENT RESSOURCE

Document ressource

Cours 0.1

v(None)

IUT d'Annecy, 9 rue de l'Arc en Ciel, 74940 Annecy

ETHERNET/IP

Sommaire

1	Architecture réseau	3
1.1	Couches protocolaires	3
1.2	Rôles Scanner / Adapter	3
1.3	Modèle producteur/consommateur	3
2	Modèle objet CIP	4
2.1	Objets, instances, attributs, services	4
2.2	Objets standards essentiels	4
3	Modes de communication	5
3.1	Messagerie explicite	5
3.2	Messagerie implicite	6
4	Objet Assembly	6
4.1	Les trois assemblies typiques	6
4.2	Dimensionnement	7
5	Mise en œuvre côté Codesys	7
5.1	Les briques de configuration	7
5.2	Où placer la couche objet applicative	8
6	Mise en œuvre sur un automate Schneider M340 (Control Expert)	8
6.1	Support matériel EtherNet/IP sur M340	9
6.2	Démarche de configuration dans Control Expert	9
6.3	Messagerie explicite depuis un M340	9
6.4	Où placer la couche objet applicative	10
7	Checklist de conception	10



8	Tableau récapitulatif et glossaire	11
8.1	Objets CIP standards à connaître	11
8.2	Glossaire	11

Document ressource – Aide-mémoire de conception

Ce document est une ressource technique, pas un TP corrigé. Il rassemble les notions nécessaires pour **concevoir vous-même**, en TP, un système de communication EtherNet/IP entre un automate Codesys et un ou plusieurs équipements de terrain, intégré à une architecture logicielle orientée objet. Consultez-le pendant votre conception comme un aide-mémoire, pas comme un cours à lire linéairement.

EtherNet/IP dans la famille CIP

EtherNet/IP (Ethernet Industrial Protocol) est un protocole de communication industriel qui transporte le protocole applicatif **CIP** (*Common Industrial Protocol*) sur une infrastructure Ethernet standard (TCP/IP et UDP/IP). CIP est un protocole *orienté objet* : tout équipement, toute donnée échangée est représenté par des objets, des instances, des attributs et des services – une philosophie proche de celle que vous manipulez déjà en programmation orientée objet sous Codesys.

CIP est également le protocole applicatif d'autres réseaux industriels (DeviceNet, ControlNet). EtherNet/IP n'est donc qu'un *transport* particulier (Ethernet) pour ce même modèle objet.



Pourquoi choisir EtherNet/IP ?

EtherNet/IP est largement utilisé dans les cellules robotisées et les lignes de production automatisées, en particulier avec des équipements Rockwell/Allen-Bradley, des robots industriels (ABB, FANUC, Kuka, Stäubli...) ou des caméras de vision industrielle. Il permet à la fois des échanges cycliques rapides (E/S temps réel) et des échanges ponctuels riches (diagnostic, paramétrage, lecture d'attributs).

Positionnement par rapport aux autres protocoles industriels

Critère	Modbus TCP	PROFINET	EtherNet/IP
Modèle de données	Registres bruts (adresses numériques)	Objets et modules GSD	Objets CIP (instances, attributs, services)
Échange cyclique	Requête/réponse maître-esclave, pas de connexion établie	Connexion RT établie, cyclique	Connexion établie (<i>implicit messaging</i>), producteur/consommateur
Échange ponctuel	Lecture/écriture de registres	Acyclique (diagnostic, alarmes)	Messagerie explicite (services Get/Set Attribute)
Origine industrielle	Générique, très répandu	Siemens / PROFIBUS & PROFINET International	Rockwell / ODVA
Complexité de mise en œuvre	Faible	Moyenne (fichiers GSDML)	Moyenne (fichiers EDS, modèle objet)



Ce que vous devez retenir

Contrairement à Modbus TCP (simple lecture/écriture de registres numérotés, sans sémantique), EtherNet/IP structure les données autour d'**objets** identifiés par une classe, une instance et des attributs. Cette organisation est directement transposable à la conception orientée objet de votre application Codesys : chaque équipement de terrain peut être représenté, côté automate, par un objet logiciel qui encapsule sa communication CIP.

1 Architecture réseau

1.1 Couches protocolaires

EtherNet/IP s'appuie sur les couches standards Ethernet/TCP/IP et ajoute une couche applicative CIP :

Couche	Rôle
CIP (application)	Modèle objet : classes, instances, attributs, services. Commun à tous les réseaux CIP.
TCP ou UDP (transport)	TCP pour la messagerie explicite (connexion, fiabilité). UDP pour la messagerie implicite (rapidité, cyclique).
IP (réseau)	Adressage IP standard, routable.
Ethernet (liaison/physique)	Infrastructure Ethernet standard (commutateurs, câblage RJ45, éventuellement industriel M12).



Un réseau Ethernet standard

EtherNet/IP n'impose pas de matériel réseau spécifique : commutateurs (switchs) Ethernet standards, câblage cuivre ou fibre. C'est un avantage par rapport à des bus de terrain propriétaires, mais cela implique aussi de maîtriser les problématiques classiques des réseaux Ethernet (collisions, latence, tempête de broadcast) dans un contexte industriel temps réel.

1.2 Rôles Scanner / Adapter

Dans une relation de communication EtherNet/IP, deux rôles sont clairement identifiés :

Scanner (Originator)

- Initie la connexion.
- Ouvre les connexions implicites (cycliques) vers un ou plusieurs Adapters.
- Peut également émettre des requêtes de messagerie explicite.
- Typiquement : l'automate (ex. Codesys) qui pilote la cellule.

Adapter (Target)

- Équipement de terrain qui répond aux connexions établies par un Scanner.
- Expose des objets CIP (dont des *Assembly*) consultables/paramétrables.
- Typiquement : un variateur, une E/S déportée, une caméra, un robot.



Un même automate peut jouer les deux rôles

Un automate Codesys configuré comme Scanner EtherNet/IP peut, dans certaines architectures, également exposer des données en tant qu'Adapter vis-à-vis d'un superviseur ou d'un autre automate. Il faut donc toujours préciser, pour *chaque* liaison de votre architecture, qui est Scanner et qui est Adapter – ce n'est pas une propriété globale de l'équipement.

1.3 Modèle producteur/consommateur

EtherNet/IP repose sur un modèle *producteur/consommateur* (*producer/consumer*) plutôt que sur un simple modèle maître/esclave point à point :

- Un **producteur** publie une donnée sur le réseau (par exemple via une connexion multicast) sans se soucier explicitement de qui la consomme.
- Un ou plusieurs **consommateurs** s'abonnent à cette donnée.
- Ce modèle permet à plusieurs équipements de recevoir la même donnée produite une seule fois, avec une charge réseau maîtrisée.

Ce modèle est complété par un modèle *maître/esclave classique* pour la partie messagerie explicite, où le Scanner interroge directement un Adapter précis (requête/réponse point à point).



Le Scanner est-il toujours consommateur ?

Non. « Scanner » et « Adapter » désignent qui *initie* la connexion, pas qui produit ou consomme la donnée. Sur une connexion implicite (cyclique) établie par le Scanner, les deux sens de l'échange coexistent :

- l'**Adapter produit** l'Input Assembly (ses mesures/états) – le **Scanner le consomme** ;
- le **Scanner produit** l'Output Assembly (ses commandes/consignes) – l'**Adapter le consomme**.

Le Scanner est donc simultanément producteur (des sorties qu'il envoie) et consommateur (des entrées qu'il reçoit) sur une même connexion. Ne confondez pas le rôle *Scanner/Adapter* (qui ouvre la connexion) avec le rôle *producteur/consommateur* (qui émet quelle donnée) : ce sont deux classifications indépendantes qui se combinent.

2 Modèle objet CIP

2.1 Objets, instances, attributs, services

Le modèle CIP structure toute donnée ou fonctionnalité d'un équipement autour de quatre notions, à mettre directement en parallèle avec la programmation orientée objet que vous pratiquez en Codesys :

Notion CIP	Définition	Analogie POO (Codesys)
Classe (<i>Class</i>)	Type d'objet générique, identifié par un code hexadécimal (ex. 0x01 = Identity).	Une classe (ex. FUNCTION_BLOCK)
Instance	Occurrence concrète d'une classe au sein d'un équipement, numérotée.	Une instance (objet) de la classe
Attribut	Donnée membre de l'instance (lisible et/ou modifiable), identifiée par un numéro.	Une propriété / variable membre
Service	Action exécutable sur une classe ou une instance (ex. <code>Get_Attribute_Single</code>).	Une méthode



Adressage d'une donnée CIP

Pour accéder à une donnée CIP, il faut donc toujours préciser un triplet : **Classe – Instance – Attribut** (souvent noté **Class/Instance/Attribute** ou **C/I/A**), auquel s'ajoute le **service** demandé (lecture, écriture, action). C'est l'équivalent CIP d'un appel de méthode ou d'accès à une propriété sur un objet logiciel précis.

2.2 Objets standards essentiels

L'ODVA (organisation qui normalise CIP) définit des objets standards, présents sur la quasi-totalité des équipements EtherNet/IP. Les connaître permet de diagnostiquer un équipement même sans documentation constructeur détaillée.

Code	Nom	Rôle
0x01	Identity Object	Identification de l'équipement : fabricant, type de produit, numéro de série, révision, état de l'équipement. Premier objet à consulter pour un diagnostic.
0x04	Assembly Object	Regroupe plusieurs attributs d'autres objets en un seul bloc de données, pour l'échange cyclique (I/O). Voir 4.
0x06	Connection Manager Object	Gère l'ouverture, la configuration et la fermeture des connexions CIP (implicites et explicites).
0xF5	TCP/IP Interface Object	Configuration de l'interface TCP/IP de l'équipement (adresse IP, masque, passerelle).
0xF6	Ethernet Link Object	Informations et statistiques de la liaison Ethernet physique (vitesse, duplex, compteurs d'erreurs).



Objets spécifiques au domaine applicatif

Au-delà de ces objets standards communs à tous les équipements, chaque famille de produits ajoute des objets métiers (ex. objet Moteur pour un variateur, objet Capteur pour une caméra de vision). Ces objets spécifiques sont documentés dans le fichier **EDS** (*Electronic Data Sheet*) fourni par le constructeur – voir le glossaire.

3 Modes de communication

CIP propose deux modes de messagerie, aux usages complémentaires.

3.1 Messagerie explicite

La **messagerie explicite** (*explicit messaging*) fonctionne sur le mode requête/réponse, typiquement sur TCP. Le Scanner envoie une requête ciblant précisément un triplet Classe/Instance/Attribut d'un objet, avec un service donné, et l'Adapter renvoie une réponse.

Service CIP	Usage
Get_Attribute_Single	Lire la valeur d'un attribut précis d'une instance.
Set_Attribute_Single	Écrire la valeur d'un attribut précis d'une instance.
Get_Attribute_All	Lire l'ensemble des attributs d'une instance en une requête.
Reset	Service générique de réinitialisation (souvent sur l'objet Identity).



Caractéristiques de la messagerie explicite

- Non cyclique : émise ponctuellement, à la demande du programme.
- Fiable (TCP), mais latence variable, non déterministe.
- Adaptée au diagnostic, au paramétrage, à la lecture ponctuelle d'informations (identité, état, configuration).
- Correspond, côté conception objet, à un appel de méthode explicite sur un objet distant.

3.2 Messagerie implicite

La **messagerie implicite** (*implicit messaging*, ou I/O cyclique) repose sur une **connexion établie** entre Scanner et Adapter, configurée une fois puis maintenue automatiquement. Les données sont échangées de façon cyclique et automatique, sans qu'il soit nécessaire de reformuler une requête à chaque échange.



RPI – Requested Packet Interval

Le **RPI** (*Requested Packet Interval*) est la période, définie à la configuration de la connexion, à laquelle les données sont produites/rafraîchies sur le réseau (exprimée en millisecondes, ex. 10 ms, 50 ms). Un RPI trop faible sur de nombreuses connexions peut saturer la bande passante réseau ou la charge CPU de l'équipement ; un RPI trop élevé dégrade la réactivité du système. Le choix du RPI est un compromis à documenter explicitement dans votre conception.

Critère	Messagerie explicite	Messagerie implicite
Transport	TCP	UDP (généralement)
Cyclicité	Ponctuelle, à la demande	Cyclique, automatique (RPI)
Contenu	Un attribut ciblé, ou une réponse structurée	Bloc de données Assembly
Usage typique	Diagnostic, paramétrage	Échange d'E/S temps réel



À retenir pour la conception

En pratique : les données qui doivent circuler en continu et rapidement (états, mesures, commandes) passent par la **messagerie implicite** via un Assembly Object dimensionné à l'avance. Les données consultées ou modifiées ponctuellement (identification, configuration, diagnostic approfondi) passent par la **messagerie explicite**. Une architecture bien conçue combine les deux.

4 Objet Assembly

L'**Assembly Object** (classe 0x04) est l'objet central de la messagerie implicite : il regroupe, dans un seul bloc de données contigu, plusieurs attributs qui seraient sinon dispersés dans plusieurs objets/instances. Cela évite de multiplier les connexions cycliques et permet un échange efficace en une seule trame.

4.1 Les trois assemblies typiques

Assembly	Sens du point de vue de l'Adapter	Contenu typique
Input Assembly	Adapter → Scanner	États, mesures, retours de capteurs, codes de défaut.
Output Assembly	Scanner → Adapter	Commandes, consignes, ordres de marche.
Configuration Assembly	Scanner → Adapter (généralement à l'ouverture de connexion)	Paramètres de configuration transmis une fois à l'établissement de la connexion.



Point de vue Scanner vs point de vue Adapter

Attention au sens : ce que l'Adapter appelle son *Input Assembly* correspond, côté Scanner (votre automate), à une donnée reçue en **entrée**. Ce que l'Adapter appelle son *Output Assembly* correspond, côté Scanner, à une donnée envoyée en **sortie**. La documentation constructeur (fichier EDS) se place presque toujours du point de vue de l'Adapter – gardez ce repère en tête au moment de câbler vos variables Codesys.

4.2 Dimensionnement

Chaque Assembly a une **taille fixe**, exprimée en octets, définie soit par une instance standard proposée par l'équipement, soit par une configuration (*assembly configurable*) lorsque l'équipement le permet. Ce dimensionnement doit être décidé *avant* la mise en œuvre :

- Lister précisément les données à échanger dans chaque sens.
- Choisir un format compact (bits groupés en octets/mots, éviter le gaspillage).
- Vérifier la correspondance entre la taille de l'Assembly déclarée côté Adapter et celle attendue côté configuration Scanner : une incohérence empêche l'établissement de la connexion.



Lien avec votre code orienté objet

Une bonne pratique de conception consiste à faire correspondre un Assembly Object à une **structure de données** (ou à une classe simple) côté Codesys, représentant l'état échangé avec l'équipement distant. L'objet logiciel qui pilote l'équipement (ex. *CVariableur*, *CCameraVision*) encapsule alors la mise en forme/lecture de cette structure, et expose à son tour des propriétés et méthodes de haut niveau au reste de l'application – la complexité du mapping CIP reste cachée à l'intérieur de l'objet.

5 Mise en œuvre côté Codesys

Cette partie reste volontairement générique : elle décrit les briques et la démarche, sans dépendre d'un équipement particulier, afin que vous puissiez l'appliquer à n'importe quel Adapter EtherNet/IP rencontré en TP.

5.1 Les briques de configuration

Dans l'arbre de configuration matérielle Codesys, la mise en œuvre d'une liaison EtherNet/IP repose généralement sur :

- Un **scanner EtherNet/IP** ajouté sous l'interface Ethernet de l'automate (rôle Scanner/Originator).
- Un ou plusieurs **devices Adapter**, ajoutés sous ce scanner, décrits par un fichier **EDS** fourni par le constructeur (ou un device générique si aucun EDS n'est disponible).
- Pour chaque device : la configuration de sa/ses **connexion(s)** (choix de l'Assembly Input/Output/Config utilisé, taille, RPI).
- Le **mapping** des variables d'E/S automate sur les octets/mots de chaque Assembly.



Où se situe la couche objet CIP ?

La couche objet CIP (classes, instances, attributs, Assembly) est gérée par le **runtime EtherNet/IP** de Codesys : ouverture/maintien des connexions, sérialisation des Assembly en trames réseau, gestion du RPI. Ce que voit votre programme ST, ce sont des **variables mappées** (issues du scanner) qui reflètent le contenu des Assembly – des octets, mots, bits bruts. La sémantique CIP (quel octet représente quel attribut) est définie une fois pour toutes à la configuration, pas dans le code applicatif.

5.2 Où placer la couche objet applicative

La conception orientée objet de votre application intervient *au-dessus* du mapping brut fourni par le scanner : c'est à vous de construire l'abstraction qui donne du sens aux octets échangés.



Principe d'encapsulation appliqué à un équipement EtherNet/IP

Une classe dédiée à un équipement CIP (ex. CAdapterEthernetIP ou une classe spécifique comme CVariateurAxe) peut :

- recevoir, en entrée de sa méthode de mise à jour cyclique, les variables brutes issues du mapping Input Assembly;
- exposer des **propriétés** de haut niveau (ex. Vitesse, EtatDefault, PositionCourante) qui décodent ces octets bruts;
- exposer des **méthodes** qui construisent le contenu attendu de l'Output Assembly (ex. Demarrer(), ConsigneVitesse(...));
- encapsuler, si nécessaire, l'émission de requêtes de messagerie explicite pour du diagnostic ponctuel (lecture de l'objet Identity, par exemple).

Un squelette de structure de communication (extrait de principe, à adapter) :

```

FUNCTION_BLOCK CAdapterEthernetIP
VAR
    m_diBrutIn  : DWORD;    // reflet brut de l'Input Assembly (mapping scanner)
    m_diBrutOut : DWORD;    // reflet brut de l'Output Assembly (mapping scanner)
5    m_xDefault : BOOL;
END_VAR

// Méthode appelée cycliquement, en lien avec le mapping du scanner
METHOD MActualiser
VAR_INPUT
10    diEntreeBrute : DWORD;
END_VAR
    m_diBrutIn := diEntreeBrute;
    m_xDefault := m_diBrutIn.0; // exemple : bit 0 = défaut
15 END_METHOD

// Propriété de haut niveau construite à partir des données brutes
PROPERTY EtatDefault : BOOL
    GET
20     EtatDefault := m_xDefault;
    END_GET
END_PROPERTY

// Méthode qui prépare la sortie à transmettre au scanner
25 METHOD MDemarrer
    m_diBrutOut.0 := TRUE; // exemple : bit 0 = commande marche
END_METHOD
  
```

Listing 1 – Squelette d'une classe encapsulant un échange cyclique via Assembly



Ce que ce squelette n'impose pas

Ce squelette illustre uniquement le *principe* d'encapsulation : la répartition exacte des bits/octets dépend entièrement de la définition de l'Assembly de l'équipement réellement utilisé en TP (voir sa documentation EDS). Ne recopiez pas ce code tel quel : concevez la structure de vos propres classes à partir de l'Assembly réel de votre Adapter.

6 Mise en œuvre sur un automate Schneider M340 (Control Expert)

Cette partie transpose la démarche générique de la section 5 à un automate **Modicon M340** programmé avec **Control Expert** (ex-Unity Pro). Les concepts CIP (Scanner, Adapter, Assembly, RPI) sont strictement identiques – seuls l'outil de configuration et le vocabulaire de mapping changent.

6.1 Support matériel EtherNet/IP sur M340

Sur la gamme M340, le rôle de Scanner EtherNet/IP est porté par un **module de communication dédié** (BMX NOE 0100/0110), à ajouter dans le rack. Le port Ethernet intégré du processeur (CPU BMX P34 2020/2030/2040) reste, quant à lui, réservé aux services Modbus TCP, au serveur web de diagnostic (FactoryCast) et aux échanges Global Data – il ne réalise pas de scan EtherNet/IP.



Un service parmi d'autres sur le module NOE

Le module BMX NOE est lui aussi multi-services : messagerie Modbus TCP, serveur web de diagnostic, et *scanner EtherNet/IP* peuvent coexister sur la même interface selon le firmware installé. Activer le scanner EtherNet/IP ne désactive pas les autres services – il faut seulement veiller à la charge réseau globale et vérifier, dans la documentation Schneider du module, que le firmware installé supporte bien la fonction de scan EtherNet/IP.

6.2 Démarche de configuration dans Control Expert

La configuration d'une liaison EtherNet/IP s'effectue dans l'éditeur de réseau (*DTM Network Editor / configuration matérielle*) :

1. **Déclarer le scanner EtherNet/IP** sur le module de communication BMX NOE : c'est l'équivalent du scanner ajouté sous l'interface Ethernet dans Codesys.
2. **Importer le fichier EDS** de l'Adapter cible dans le catalogue de matériel (*Hardware Catalog*), s'il n'y est pas déjà – ou utiliser un DTM constructeur si l'équipement en fournit un.
3. **Ajouter le device Adapter** sous le scanner, en renseignant son adresse IP.
4. **Configurer la connexion I/O scanning** : choix de l'Assembly Input/Output (souvent proposé par des tailles prédéfinies dans l'EDS), et **RPI** (période de rafraîchissement, en ms).
5. **Mapper les données** sur des zones mémoire automate (%MW, %IW, ou variables DDT structurées), qui seront ensuite manipulées dans le programme (Ladder, ST, ou blocs fonctionnels).



Correspondance avec le vocabulaire Codesys

Codesys	Control Expert
Scanner ajouté sous l'interface Ethernet	Scanner EtherNet/IP configuré sur le module BMX NOE
Device Adapter + fichier EDS	Équipement ajouté au catalogue matériel via EDS/DTM
Variables mappées sur l'Input/Output Assembly	Zone %MW (ou DDT) associée à la connexion I/O scanning
Programme ST au-dessus du mapping	Programme Ladder/ST/FBD au-dessus des %MW

6.3 Messagerie explicite depuis un M340

Le module BMX NOE ne se limite pas à l'échange cyclique (implicite) : il permet aussi d'émettre des **requêtes explicites CIP** depuis le programme Control Expert, par exemple pour lire l'objet Identity d'un équipement au démarrage, ou déclencher ponctuellement un service qui ne justifie pas une connexion cyclique dédiée.

1. **Fonction générique DATA_EXCH** : ce bloc fonction, déjà utilisé pour la messagerie explicite Modbus TCP, peut également construire et envoyer une requête CIP explicite (service, classe, instance, attribut) vers l'adresse IP de l'Adapter, puis récupérer la réponse dans une table de gestion (mots de statut/échange).
2. **Bloc dédié de la *General Purpose Library* (GPL)** : Control Expert propose, dans sa bibliothèque de fonctions génériques, un bloc spécialisé pour la messagerie explicite EtherNet/IP, où le service CIP,

la classe, l'instance et l'attribut visés se renseignent directement sur des entrées du bloc – plus simple à mettre en œuvre que le paramétrage bas niveau de `DATA_EXCH`.



Un mécanisme asynchrone, comme sous Codesys

Comme toute messagerie explicite CIP, l'échange est asynchrone : le programme déclenche la requête, puis doit scruter un mot d'état/de gestion (souvent appelé *rapport d'échange*) pour savoir quand la réponse est disponible, si elle a réussi, et récupérer les données reçues. Ne bloquez jamais un cycle automate en attendant activement la fin de l'échange.



Explicite et implicite se partagent le même NOE

Multiplier les requêtes explicites sur un module déjà chargé par plusieurs connexions I/O scanning peut dégrader les temps de réponse de l'ensemble. Réservez la messagerie explicite aux échanges ponctuels (diagnostic, identification, paramétrage) et laissez les données cycliques temps réel transiter par les Assembly de la connexion implicite – exactement la même règle de conception qu'à la section 3 sous Codesys.

6.4 Où placer la couche objet applicative

Le principe d'encapsulation vu à la section 5 reste valable : les %MW issus du scanner M340 ne sont que le reflet brut de l'Assembly CIP échangé. Rien n'empêche, en ST sous Control Expert (types DDT + blocs fonctionnels), de reproduire la même logique de classes/propriétés/méthodes pour donner du sens à ces mots mémoire avant de les exposer au reste de l'application – la plate-forme change, pas la démarche de conception.



Diagnostic de la connexion

Control Expert expose, pour chaque connexion I/O scanning configurée, un état de santé (souvent un mot ou quelques bits dédiés, parfois nommé *Health Word* selon l'EDS de l'équipement) permettant de détecter une perte de connexion avec l'Adapter (câble débranché, adresse IP erronée, équipement hors tension). Intégrez systématiquement ce diagnostic dans votre classe d'encapsulation – une valeur figée dans l'Input Assembly peut aussi bien signifier « rien ne change » que « la connexion est rompue ».

7 Checklist de conception



À décider avant de commencer le TP

Avant d'ouvrir Codesys, formalisez par écrit les points suivants pour votre système. Cette fiche peut être complétée directement et conservée comme mémo de conception.

1. **Rôles** : quel équipement est Scanner ? Quel(s) équipement(s) sont Adapter(s) ? (1)
2. **Adressage** : quelle adresse IP pour chaque équipement ? Sont-elles sur le même sous-réseau ?
3. **Données échangées** : lister précisément, pour chaque Adapter, les données à transmettre du Scanner vers l'Adapter (commandes) et de l'Adapter vers le Scanner (états/mesures).
4. **Dimensionnement des Assembly** : taille en octets de l'Input Assembly, de l'Output Assembly, et de la Configuration Assembly le cas échéant. (4)
5. **RPI** : quelle période de rafraîchissement cyclique choisir pour chaque connexion, en cohérence avec la dynamique du procédé et la charge réseau tolérable ? (3)
6. **Messagerie explicite** : quelles informations ne nécessitent qu'une lecture/écriture ponctuelle (identification, diagnostic, paramétrage) plutôt qu'un échange cyclique ?
7. **Gestion des erreurs de connexion** : comment le programme détecte-t-il une perte de connexion avec un Adapter (bit de statut du scanner, timeout) ? Quel comportement de repli adopter (arrêt sécurisé, dernière valeur connue, signalisation) ?

8. **Découpage objet** : quelle classe (ou quel ensemble de classes) va encapsuler la communication avec chaque équipement ? Quelles propriétés et méthodes seront exposées au reste de l'application ? (5)
9. **Documentation constructeur** : le fichier EDS de l'équipement est-il disponible ? Quels objets/Assembly spécifiques au métier de l'équipement propose-t-il, au-delà des objets standards ?



Erreurs fréquentes à anticiper

- Confondre le sens Input/Output entre le point de vue de l'Adapter (documentation constructeur) et celui du Scanner (votre programme).
- Ne pas gérer explicitement la perte de connexion : un Adapter déconnecté peut laisser des données figées sur leur dernière valeur si aucune détection n'est mise en place.
- Sous-dimensionner ou surdimensionner un Assembly par rapport aux besoins réels, rendant le mapping confus ou gaspillant de la bande passante.
- Placer la logique métier directement sur les variables brutes du mapping, au lieu de l'encapsuler dans une classe dédiée – ce qui rend le code difficile à faire évoluer si l'équipement change.

8 Tableau récapitulatif et glossaire

8.1 Objets CIP standards à connaître

Code	Objet	Usage principal
0x01	Identity Object	Identification de l'équipement, diagnostic de base.
0x04	Assembly Object	Regroupement de données pour l'échange cyclique.
0x06	Connection Manager Object	Gestion des connexions CIP.
0xF5	TCP/IP Interface Object	Configuration réseau IP.
0xF6	Ethernet Link Object	Statistiques et état de la liaison Ethernet.

8.2 Glossaire

CIP *Common Industrial Protocol* – protocole applicatif orienté objet commun à EtherNet/IP, DeviceNet et ControlNet.

Scanner (Originator) Équipement qui initie les connexions CIP ; typiquement l'automate maître.

Adapter (Target) Équipement qui répond aux connexions établies par un Scanner ; typiquement un équipement de terrain.

Assembly Object Objet CIP (classe 0x04) regroupant plusieurs attributs en un bloc de données unique, pour l'échange cyclique.

Messagerie explicite Échange CIP requête/réponse, ponctuel, généralement sur TCP.

Messagerie implicite Échange CIP cyclique via une connexion établie, généralement sur UDP.

RPI *Requested Packet Interval* – période de rafraîchissement d'une connexion cyclique.

EDS *Electronic Data Sheet* – fichier texte normalisé fourni par le constructeur, décrivant les objets, Assembly et paramètres d'un équipement CIP.

Producteur/consommateur Modèle d'échange où une donnée est publiée une fois par un producteur et reçue par un ou plusieurs consommateurs.

ODVA *Open DeviceNet Vendors Association* – organisation qui édite et maintient les standards CIP/EtherNet/IP.

Class/Instance/Attribute (C/I/A) Triplet identifiant de façon unique une donnée CIP au sein d'un équipement.